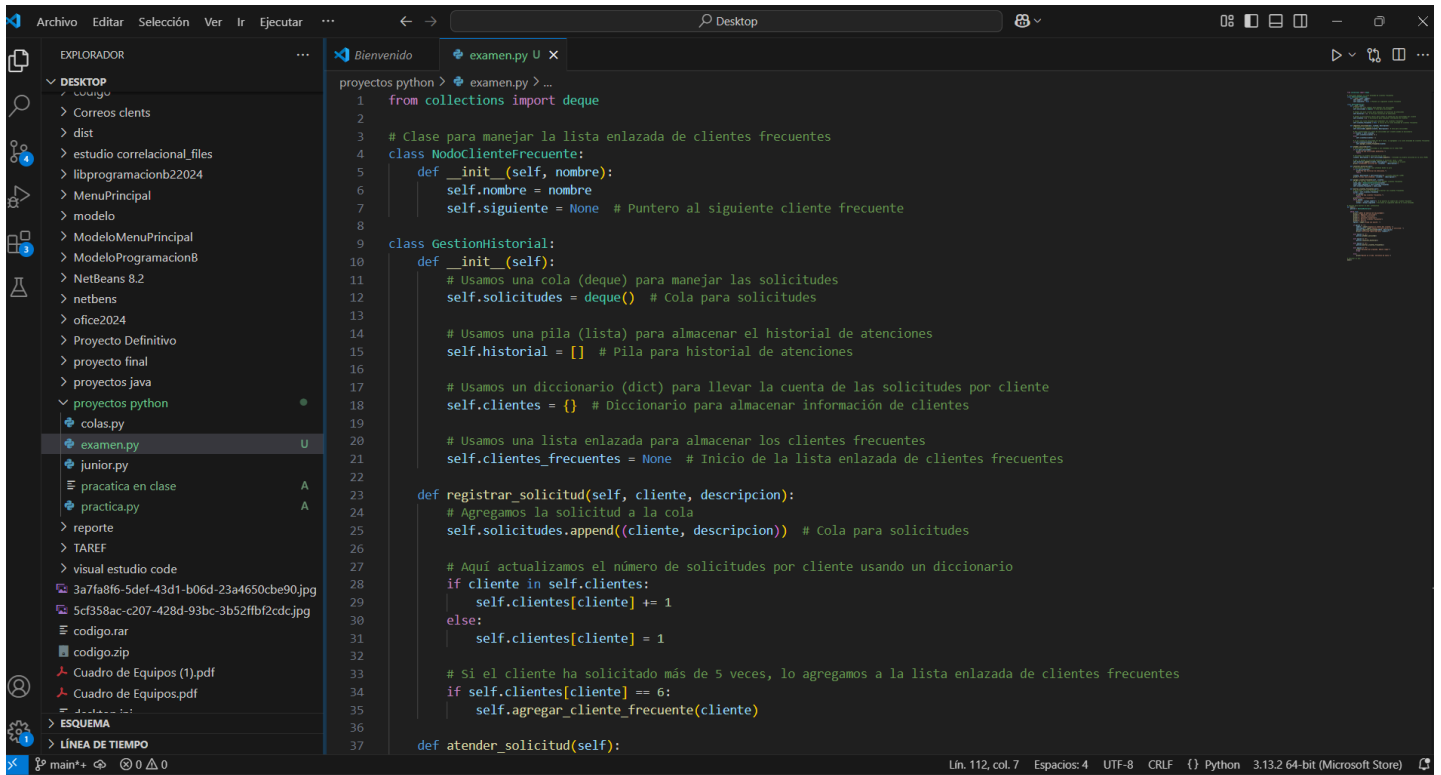
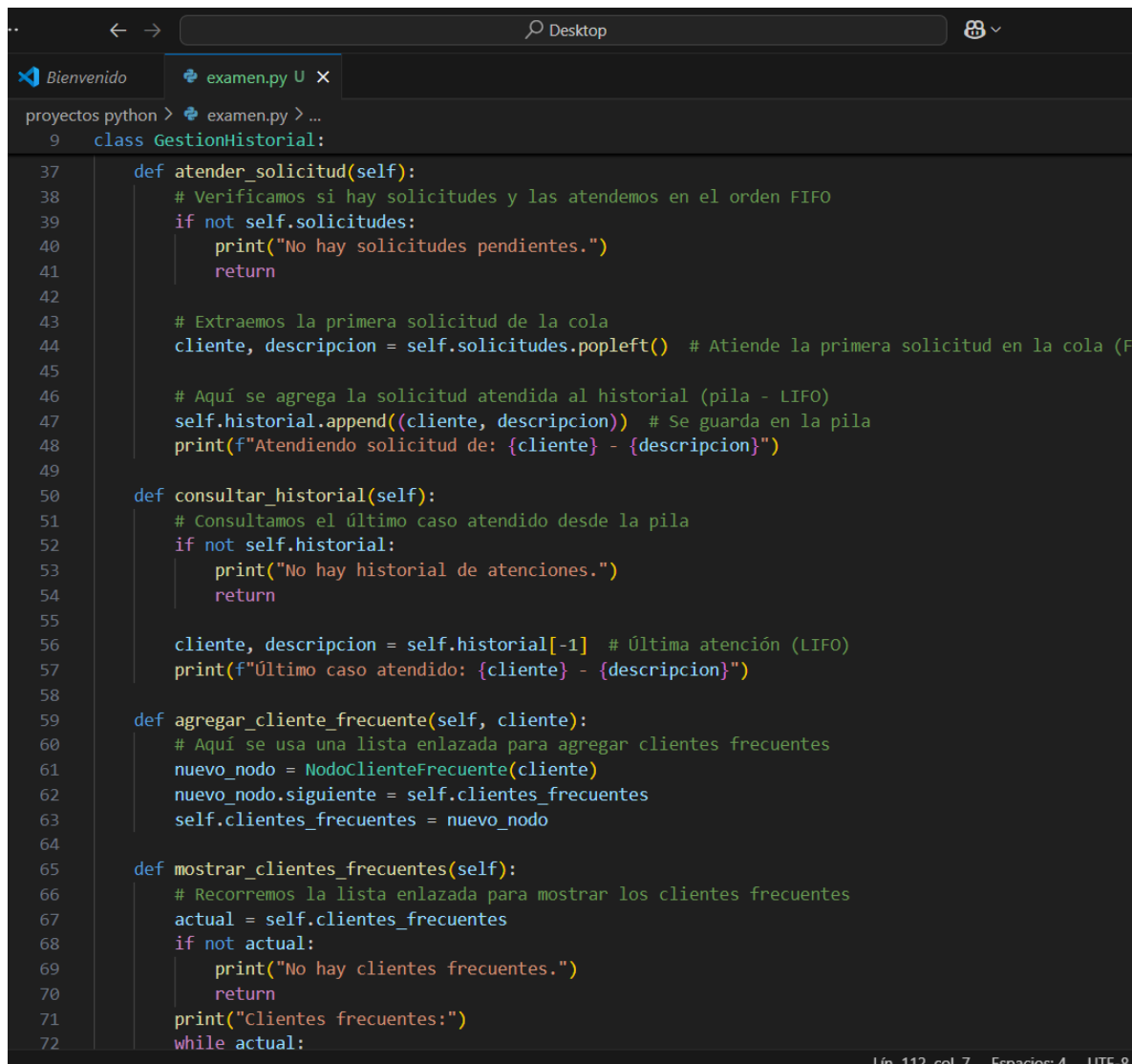


Código fuente junior Fernando gomez cano



The screenshot shows the VS Code interface with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like 'Correos clnts', 'dist', 'estudio correlacional_files', 'libprogramacionb22024', 'MenuPrincipal', 'modelo', 'ModeloMenuPrincipal', 'ModeloProgramacionB', 'NetBeans 8.2', 'netbens', 'office2024', 'Proyecto Definitivo', 'proyecto final', 'proyectos java', 'proyectos python', 'colas.py', 'examen.py', 'junior.py', 'practica en clase', 'practica.py', 'reporte', 'TAREF', 'visual estudio code', and files like '3a7fa8f6-5def-43d1-b06d-23a4650cbe90.jpg', '5cf358ac-c207-428d-93bc-3b52fbbf2cdc.jpg', 'codigo.rar', 'codigo.zip', 'Cuadro de Equipos (1).pdf', and 'Cuadro de Equipos.pdf'. The code editor shows the following code:

```
1 from collections import deque
2
3 # Clase para manejar la lista enlazada de clientes frecuentes
4 class NodoClienteFrecuente:
5     def __init__(self, nombre):
6         self.nombre = nombre
7         self.siguiente = None # Puntero al siguiente cliente frecuente
8
9 class GestionHistorial:
10     def __init__(self):
11         # Usamos una cola (deque) para manejar las solicitudes
12         self.solicitudes = deque() # Cola para solicitudes
13
14         # Usamos una pila (lista) para almacenar el historial de atenciones
15         self.historial = [] # Pila para historial de atenciones
16
17         # Usamos un diccionario (dict) para llevar la cuenta de las solicitudes por cliente
18         self.clientes = {} # Diccionario para almacenar información de clientes
19
20         # Usamos una lista enlazada para almacenar los clientes frecuentes
21         self.clientes_frecuentes = None # Inicio de la lista enlazada de clientes frecuentes
22
23     def registrar_solicitud(self, cliente, descripcion):
24         # Agregamos la solicitud a la cola
25         self.solicitudes.append((cliente, descripcion)) # Cola para solicitudes
26
27         # Aquí actualizamos el número de solicitudes por cliente usando un diccionario
28         if cliente in self.clientes:
29             self.clientes[cliente] += 1
30         else:
31             self.clientes[cliente] = 1
32
33         # Si el cliente ha solicitado más de 5 veces, lo agregamos a la lista enlazada de clientes frecuentes
34         if self.clientes[cliente] == 6:
35             self.agregar_cliente_frecuente(cliente)
36
37     def atender_solicitud(self):
```



The continuation of the Python code from the previous screenshot, showing methods for handling requests, consulting history, adding frequent clients, and displaying frequent clients.

```
37     def atender_solicitud(self):
38         # Verificamos si hay solicitudes y las atendemos en el orden FIFO
39         if not self.solicitudes:
40             print("No hay solicitudes pendientes.")
41             return
42
43         # Extraemos la primera solicitud de la cola
44         cliente, descripcion = self.solicitudes.popleft() # Atiende la primera solicitud en la cola (FIFO)
45
46         # Aquí se agrega la solicitud atendida al historial (pila - LIFO)
47         self.historial.append((cliente, descripcion)) # Se guarda en la pila
48         print(f"Atendiendo solicitud de: {cliente} - {descripcion}")
49
50     def consultar_historial(self):
51         # Consultamos el último caso atendido desde la pila
52         if not self.historial:
53             print("No hay historial de atenciones.")
54             return
55
56         cliente, descripcion = self.historial[-1] # Última atención (LIFO)
57         print(f"Último caso atendido: {cliente} - {descripcion}")
58
59     def agregar_cliente_frecuente(self, cliente):
60         # Aquí se usa una lista enlazada para agregar clientes frecuentes
61         nuevo_nodo = NodoClienteFrecuente(cliente)
62         nuevo_nodo.siguiente = self.clientes_frecuentes
63         self.clientes_frecuentes = nuevo_nodo
64
65     def mostrar_clientes_frecuentes(self):
66         # Recorremos la lista enlazada para mostrar los clientes frecuentes
67         actual = self.clientes_frecuentes
68         if not actual:
69             print("No hay clientes frecuentes.")
70             return
71         print("Clientes frecuentes:")
72         while actual:
```

```
Bienvenido | examen.py U X
proyectos python > examen.py > ...
9   class GestionHistorial:
65   def mostrar_clientes_frecuentes(self):
72       while actual:
73           print(f"- {actual.nombre}") # Se muestra el nombre del cliente frecuente
74           actual = actual.siguiente # Se mueve al siguiente nodo de la lista enlazada
75
76   # Función para mostrar el menú interactivo
77   def menu():
78       gestion = GestionHistorial()
79
80       while True:
81           print("\nMENÚ DE GESTIÓN DE SOLICITUDES")
82           print("1. Registrar solicitud")
83           print("2. Atender solicitud")
84           print("3. Consultar historial")
85           print("4. Mostrar clientes frecuentes")
86           print("5. Salir")
87           opcion = input("Elige una opción: ")
88
89           if opcion == "1":
90               nombre = input("Ingrese el nombre del cliente: ")
91               descripcion = input("Ingrese la descripción de la solicitud: ")
92               gestion.registrar_solicitud(nombre, descripcion)
93               print(f"Solicitud registrada para {nombre}")
94
95           elif opcion == "2":
96               gestion.atender_solicitud()
97
98           elif opcion == "3":
99               gestion.consultar_historial()
100
101           elif opcion == "4":
102               gestion.mostrar_clientes_frecuentes()
103
104           elif opcion == "5":
105               print("Saliendo del programa. ¡Hasta luego!")
106               break
```

```
77   def menu():
103
104       elif opcion == "5":
105           print("Saliendo del programa. ¡Hasta luego!")
106           break
107
108       else:
109           print("Opción no válida. Inténtalo de nuevo.")
110
111   # Ejecutar el menú
112   menu()
```

Funciones del menú

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS
Python + v [icon] [icon] ... ^ X

3 Consultar historial
4 Mostrar clientes frecuentes
5 Salir
Elige una opción: & "c:/Users/Junior Gómez/AppData/Local/Microsoft/WindowsApps/python3.13.exe" "c:/Users/Junior Gómez/Desktop/proyectos python/examen.py"
⚠ Opción no válida. Inténtalo de nuevo.

🚀 MENÚ DE GESTIÓN DE SOLICITUDES
1 Registrar solicitud
2 Atender solicitud
3 Consultar historial
4 Mostrar clientes frecuentes
5 Salir
Elige una opción: [
```

Prueba

```
5 Salir
Elige una opción: 1
Ingrese el nombre del cliente: junior gomez
Ingrese la descripción de la solicitud: examen
✅ Solicitud registrada para junior gomez
```

```
3 Consultar historial
4 Mostrar clientes frecuentes
5 Salir
Elige una opción: 2
✅ Atendiendo solicitud de: juan - enfermedad
```

```
4 Mostrar clientes frecuentes
5 Salir
Elige una opción: 3
📄 Último caso atendido: juan - enfermedad
```

Donde aplique los métodos pedidos

Uso de una **Cola**: La **cola** se utiliza para manejar las solicitudes en el orden de llegada

```
class GestionHistorial:
    def __init__(self):
        # Usamos una cola (deque) para manejar las solicitudes
        self.solicitudes = deque() # Cola para solicitudes

    def registrar_solicitud(self, cliente, descripcion):
        # Agregamos la solicitud a la cola
        self.solicitudes.append((cliente, descripcion)) # Cola para solicitudes

    def atender_solicitud(self):
        # Verificamos si hay solicitudes y las atendemos en el orden FIFO
        if not self.solicitudes:
            print("No hay solicitudes pendientes.")
            return
        cliente, descripcion = self.solicitudes.popleft() # Atiende la primera solicitud en la cola (FIFO)
        self.historial.append((cliente, descripcion)) # Agrega al historial
        print(f"Atendiendo solicitud de: {cliente} - {descripcion}")
```

Código fuente junior Fernando gomez cano

Uso de pila

se usa para almacenar el historial de atenciones

```
class GestionHistorial:
    def __init__(self):
        # Usamos una pila (lista) para almacenar el historial de atenciones
        self.historial = [] # Pila para historial de atenciones

    def atender_solicitud(self):
        cliente, descripcion = self.solicitudes.popleft()
        self.historial.append((cliente, descripcion)) # Guardamos en la pila (LIFO)
        print(f"Atendiendo solicitud de: {cliente} - {descripcion}")

    def consultar_historial(self):
        # Consultamos el último elemento de la pila
        if not self.historial:
            print("No hay historial de atenciones.")
            return
        cliente, descripcion = self.historial[-1] # Última atención (LIFO)
        print(f"Último caso atendido: {cliente} - {descripcion}")
```

Uso de una Lista Enlazada

La lista enlazada se usa para almacenar a los clientes frecuentes que han solicitado soporte más de 5 veces.

```
class NodoClienteFrecuente:
    def __init__(self, nombre):
        self.nombre = nombre
        self.siguiente = None # Puntero al siguiente cliente frecuente

class GestionHistorial:
    def __init__(self):
        # Usamos una lista enlazada para gestionar los clientes frecuentes
        self.clientes_frecuentes = None # Inicio de la lista enlazada de clientes frecuentes

    def agregar_cliente_frecuente(self, cliente):
        # Añadimos un cliente frecuente a la lista enlazada
        nuevo_nodo = NodoClienteFrecuente(cliente)
        nuevo_nodo.siguiente = self.clientes_frecuentes
        self.clientes_frecuentes = nuevo_nodo

    def mostrar_clientes_frecuentes(self):
        # Recorremos la lista enlazada para mostrar los clientes frecuentes
        actual = self.clientes_frecuentes
        if not actual:
            print(f"No hay clientes frecuentes.")
            return
        print("Clientes frecuentes:")
        while actual:
            print(f"- {actual.nombre}") # Muestra el nombre del cliente
            actual = actual.siguiente # Se mueve al siguiente nodo
```